

VERIQO Support and Diagnostics

What to collect before raising an issue, and what the deployment can tell you on its own.

Honest note: there is **no dedicated support-bundle endpoint** — no single call that packages logs, configuration, and version for a ticket. This document describes assembling the same information from the real endpoints that do exist. Building a `GET /v1/diagnostics` bundle is a named, straightforward next step, not a designed capability being withheld.

1. Diagnostic surfaces that exist

Surface	Tells you
<code>GET /livez</code>	The HTTP server is serving. Dependency-free by design.
<code>GET /readyz</code>	Whether every configured dependency is fit to serve, with a per-check <code>detail</code> string. Returns 503 when not.
<code>GET /healthz</code>	Combined status plus whether registry persistence is enabled.
<code>GET /v1/metrics</code>	Nine real operational counters (see §3).
Process logs	Method, path, status, latency per request. No request bodies.
Startup log line	Recovery statistics from the WAL — recovered/replayed record counts.
<code>POST /v1/evidence/{id}/verify</code>	Live re-verification of one item's manifest and custody chain.
<code>GET /v1/cases/{id}/replay</code>	Whether a case's decision still re-derives to identical hashes.
<code>veriqo-commercial-verify</code>	Independent check of any exported package, offline.

All three health endpoints are exempt from the auth stack, so a probe does not need credentials.

2. Triage in order

1. Is the process alive? `GET /livez`. No response → the process is down or unreachable; check the host and the network path before anything else.

2. Is it ready? `GET /readyz`. Alive but 503 means a dependency is unfit. Read the `detail` in `checks.commercial_store` — `"store is closed"` means the durable store was closed (shutdown in progress, or a failed reopen).

3. Did it recover cleanly? Find the startup log line and its recovery statistics. Non-zero lost records, or a corrupt-middle or broken-chain finding, is an integrity matter — go to the Incident Response Procedure rather than continuing to triage normally.

4. Are the correctness counters clean? `GET /v1/metrics`. See §3.

5. Is it a specific case or item? Verify that item; replay that case. This distinguishes a single damaged record from a deployment-wide problem, which is the most useful thing to know early.

3. Reading the metrics

Nine counters. Divided by what a non-zero value means:

Must be zero. Any non-zero value is a defect, not a threshold:

Counter	Meaning
<code>evidence_verification_failures</code>	A manifest hash failed to re-derive.
<code>custody_chain_failures</code>	A custody chain failed to verify.
<code>ledger_commit_failures</code>	A write to the audit ledger failed.
<code>replay_failures</code>	A replay diverged from the recorded hashes.

Expected to move; interpret in context:

Counter	Meaning
<code>evidence_ingestion_total</code>	Volume. A flat line where you expect submissions means an upstream integration has stopped.
<code>decision_count</code> , <code>decision_latency_avg_millis</code>	Throughput and latency.
<code>authorization_denials</code>	An action was refused by the authorization gate. Some denials are the system working correctly. A sudden rise means a policy or workflow change.
<code>action_failures</code>	Execution refused after authorization.

Honest zero:

`external_adapter_failures` reads zero in every current build because no external adapters are wired in. It is an honest zero, not a healthy one — do not read it as evidence that external integrations are working.

4. What to collect before raising an issue

Attach all of the following. An issue without them will come back asking for them.

- **What you expected, what happened**, and the exact request (method, path, body with sensitive values redacted).
- **The response**: status code and the `error` string verbatim.
- **Wall-clock time** of the event, with timezone. VERIQO's ticks are logical, so only your side has real time.
- `GET /readyz` output at the time of the issue.
- `GET /v1/metrics` snapshot.
- **Process logs** spanning the event, ± 5 minutes.
- **The startup log line** from the current process, including recovery statistics.
- **Binary version identity** (release commit / source hash from the startup output).
- Deployment configuration: which auth layers are enabled, whether TLS is on, whether `commercial-wal-dir` is set. **Never send secrets** — say "JWT enabled", not the signing key.

For an integrity issue, also:

- Output of `POST /v1/evidence/{id}/verify` for affected items.

- `9744`; Output of `GET /v1/cases/{id}/replay` for affected cases.
- `9744`; Full `veriqo-commercial-verify` output for a relevant package — **including the SKIP lines**, which are diagnostic.
- `9744`; Confirmation that you have **preserved the WAL directory** per the Incident Response Procedure. Preserve before remediating.

5. Redaction

Evidence metadata routinely contains customer-sensitive values — filenames, claim and policy identifiers, vessel identities, party names, actor names.

Before sending diagnostics:

- Replace identifier *values* while keeping their *shape* (`CLM-2024-8891` → `CLM-XXXX-XXXX`). Shape often matters to diagnosis; the value rarely does.
- Never send JWT signing secrets, API keys, private key material, or a trusted-key registry's private half.
- Hashes are safe to send and usually essential.
- Request logs contain paths, which contain IDs. Redact consistently, so the same ID maps to the same placeholder throughout — otherwise the log becomes unreadable for tracing a single request.

6. Self-service checks

Before escalating, these often answer the question:

Symptom	Check
<code>403</code> on every request	Is JWT configured with no tenant membership registry? That fails closed by design — every authenticated request is refused.
<code>404</code> for a case you just created	Are you querying with the same <code>tenant_id</code> ? Cross-tenant lookups return 404, not 403.
<code>409</code> on decide	The case is already decided. Decisions are once-only by design.
<code>422</code> on decide	Cited evidence is not finalized, or the finding failed grounding. The <code>error</code> string names the gate.
<code>422</code> on dossier or replay	The case has not been decided yet.
<code>400</code> on a large request	Body limits: 1 MiB JSON, 64 MiB package upload.
Verifier reports <code>SKIP</code> on signatures	Expected without <code>-trusted-keys</code> . Not a fault.
Verifier via HTTP always skips signatures	Expected — that route passes no key registry. Use the CLI.
Writes failing after running a while	Check disk space. A write that cannot be made durable is refused rather than silently accepted.

7. Escalating an integrity issue

Integrity issues take priority over everything else, including availability. If ledger verification fails, replay diverges across multiple cases, or a WAL shows corrupt-middle or broken-chain findings:

1. Stop writes and preserve the WAL directory byte-for-byte **before** any remediation. Do not restart, do not restore.
2. Follow `VERIQO_INCIDENT_RESPONSE_PROCEDURE.md`.
3. Escalate immediately, with the preserved artifacts.

An integrity defect in VERIQO is the highest-severity class of bug this system can have, and will be treated as such.